

Winventory

BDR-Projet



Lestiboudois Maxime & Parisod Nathan & Surbeck Léon

24.01.2025

Contents

Introduction	2
Modèle EA	2
Modèle relationnel	2
Création de tables	3
Remplissage des tables	6
Description de l'application	7

Introduction

Le projet **Winventory** a pour ambition de concevoir une base de données relationnelle performante et innovante, spécialement pensée pour gérer les inventaires de vins, bières, spiritueux, boissons non alcoolisées, apéritifs et équipements.

Grâce à une solution centralisée, Winventory simplifiera la gestion des stocks, le suivi des acquisitions et les retraits de produits, tout en offrant des fonctionnalités avancées d'analyse et de reporting via une interface utilisateur intuitive.

Le système offrira une gestion centralisée des inventaires sur plusieurs magasins, rendant possible le transfert de stock entre différents sites tout en permettant une supervision globale. Par ailleurs, l'intégration des historiques d'achat des clients permettra de mieux anticiper les besoins, d'optimiser la préparation des stocks et de soutenir les actions marketing à travers des offres personnalisées et des prédictions sur les comportements d'achat. Enfin, Wineventory s'appuiera sur la robustesse de PostgreSQL pour garantir une performance optimale dans le traitement des requêtes complexes et la production de rapports détaillés. Toutes les requêtes SQL seront écrites manuellement afin de garantir un contrôle total et d'assurer une transparence complète du système.

Modèle EA

Modèle relationnel

- **Provenance**

- id, pays, région, producteur

- **Produit**

- idProduit, idProvenance, nom, tauxAlcool
- *idProvenance* référence *Provenance.id*, *idProvenance* *NOT NULL*

- **Article**

- idProduit, volume, recipient, prix, datePeremption, dateFinDeVente
- *idProduit* référence *Produit.id*

- **MouvementStock**

- id, idMagasin, idProduit, volume, recipient, date, quantite
- *idMagasin* référence *Magasin.id*
- *idProduit*, *volume*, *recipient* référence *Article.idProduit*, *volume*, *recipient*
- *idProduit*, *volume*, *recipient* *NOT NULL*

- **Magasin**

- id, nom, adresse, dateFermeture

- **Vendeur**

- id, idMagasin, nom, salaire, estActif
- *idMagasin* référence *Magasin.id*, *idMagasin NOT NULL*

- **Vente**

- idMouvementStock, idVendeur, idClient
- *idMouvementStock* référence *MouvementStock.id*
- *idVendeur* référence *Vendeur.id*, *idVendeur NOT NULL*
- *idClient* référence *Client.id*, *idClient NOT NULL*

- **Approvisionnement**

- idMouvementStock, dateCommande
- *idMouvementStock* référence *MouvementStock.id*

- **Client**

- id, nom, adresse, email, pointDeFidelite, annéeNaissance
- *email UNIQUE*

- **Fournisseur**

- id, nom, adresse, numeroTelephone

- **Approvisionnement_Fournisseur**

- idMouvementStock, idFournisseur
- *idMouvementStock* référence *MouvementStock.id*
- *idFournisseur* référence *Fournisseur.id*

Création de tables

Voici le script SQL nous permettant de créer les tables de notre base de données:

```
CREATE TABLE IF NOT EXISTS Provenance (
  id SERIAL,
  pays VARCHAR(80),
  region VARCHAR(80),
  producteur VARCHAR(80),
  CONSTRAINT PK_Provenance PRIMARY KEY (id)
);

—CREATE TYPE typeRecipient AS ENUM ('bouteille', 'canette');
```

```

CREATE TABLE IF NOT EXISTS Produit(
    idProduit SERIAL,
    idProvenance INTEGER NOT NULL,
    nom VARCHAR(80) NOT NULL,
    tauxAlcool DOUBLE PRECISION NOT NULL,
    CONSTRAINT PK_Produit PRIMARY KEY (idProduit),
    constraint FK_idProvenance foreign KEY(idProvenance) references Provenance
);

CREATE TABLE IF NOT EXISTS Article(
    idProduit INTEGER,
    volume INTEGER,
    recipient typeRecipient,
    prix DOUBLE PRECISION NOT NULL,
    datePeremption DATE NOT NULL,
    dateFinDeVente DATE,
    CONSTRAINT PK_Article PRIMARY KEY (idProduit, volume, recipient),
    CONSTRAINT FK_Article_Produit FOREIGN KEY (idProduit) REFERENCES Produit
);

CREATE TABLE IF NOT EXISTS Magasin(
    id SERIAL,
    nom VARCHAR(80) NOT NULL,
    adresse VARCHAR(350) NOT NULL,
    dateFermeture DATE,
    CONSTRAINT PK_Magasin PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS MouvementStock(
    id SERIAL,
    idMagasin INTEGER NOT NULL,
    idProduit INTEGER NOT NULL,
    volume INTEGER NOT NULL,
    recipient typeRecipient NOT NULL,
    date DATE,
    quantite INTEGER,
    CONSTRAINT PK_MouvementStock PRIMARY KEY (id),
    CONSTRAINT FK_MouvementStock_Magasin FOREIGN KEY (idMagasin) REFERENCES Magasin,
    CONSTRAINT FK_MouvementStock_Article FOREIGN KEY (idProduit, volume, recipient) REFERENCES Article
);

CREATE TABLE IF NOT EXISTS Vendeur(
    id SERIAL,
    idMagasin INTEGER NOT NULL,
    nom VARCHAR(80),

```

```

    salaire DOUBLE PRECISION,
    estActif BOOL NOT NULL,
    CONSTRAINT PK_Vendeur PRIMARY KEY (id),
    CONSTRAINT FK_Vendeur_Magasin FOREIGN KEY (idMagasin) REFERENCES Magasin
);

CREATE TABLE IF NOT EXISTS Client(
    id SERIAL,
    nom VARCHAR(80),
    adresse VARCHAR(150),
    email VARCHAR(100) UNIQUE,
    pointDeFidelite INTEGER,
    anneeNaissance INTEGER,
    CONSTRAINT PK_Client PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS Vente(
    idMouvementStock INTEGER,
    idVendeur INTEGER NOT NULL,
    idClient INTEGER NOT NULL,
    CONSTRAINT PK_Vente PRIMARY KEY (idMouvementStock),
    CONSTRAINT FK_Vente_MouvementStock FOREIGN KEY (idMouvementStock) REFERENCES MouvementStock,
    CONSTRAINT FK_Vente_Vendeur FOREIGN KEY (idVendeur) REFERENCES Vendeur,
    CONSTRAINT FK_Vente_Client FOREIGN KEY (idClient) REFERENCES Client(id)
);

CREATE TABLE IF NOT EXISTS Approvisionnement(
    idMouvementStock INTEGER,
    dateCommande DATE NOT NULL,
    CONSTRAINT PK_Approvisionnement PRIMARY KEY (idMouvementStock),
    CONSTRAINT FK_Approvisionnement_Approvisionnement FOREIGN KEY (idMouvementStock) REFERENCES Approvisionnement,
    CONSTRAINT check_DateCommande CHECK (dateCommande <= CURRENT_DATE)
);

CREATE TABLE IF NOT EXISTS Fournisseur(
    id SERIAL,
    nom VARCHAR(80),
    adresse VARCHAR(150),
    numeroTelephone VARCHAR(30),
    CONSTRAINT PK_Fournisseur PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS Approvisionnement_Fournisseur(
    idMouvementStock INTEGER,
    idFournisseur INTEGER,
    CONSTRAINT PK_Approvisionnement_Fournisseur PRIMARY KEY (idMouvementStock, idFournisseur)
);

```

```

        CONSTRAINT FK_Approvisionnement_Fournisseur_idMouvementStock FOREIGN KEY (idFournisseur, idMouvementStock)
        REFERENCES Fournisseur (idFournisseur, idMouvementStock)
    );

```

Remplissage des tables

Ce script SQL suivant nous a permis de remplir les tables de notre base de données:

```

-- Remplissage de la table Provenance
INSERT INTO Provenance (pays, region, producteur) VALUES
('France', 'Bordeaux', 'Château Margaux'),
('Italie', 'Toscane', 'Antinori'),
('Espagne', 'Ribera del Duero', 'Bodega Vega Sicilia'),
('USA', 'Napa Valley', 'Robert Mondavi'),
('France', 'Champagne', 'Moët & Chandon');

-- Remplissage de la table Produit
INSERT INTO Produit (nom, tauxAlcool, idProvenance) VALUES
('Vin Rouge Bordeaux', 13.5, 1),
('Vin Blanc Chardonnay', 12.0, 2),
('Champagne Brut', 12.5, 2),
('Whisky Single Malt', 40.0, 3),
('Bière Blonde', 5.0, 1);

-- Remplissage de la table Article
INSERT INTO Article (idProduit, volume, recipient, prix, datePeremption, dateCreation) VALUES
(1, 750, 'bouteille', 45.99, '2025-12-31', NULL),
(1, 1500, 'bouteille', 89.99, '2025-12-31', NULL),
(2, 750, 'bouteille', 35.99, '2024-11-30', '2024-10-01'),
(3, 750, 'bouteille', 55.00, '2026-06-30', NULL),
(4, 700, 'bouteille', 120.00, '2030-01-01', NULL),
(5, 330, 'canette', 2.50, '2023-12-31', '2023-11-01');

-- Remplissage de la table Magasin
INSERT INTO Magasin (nom, adresse, dateFermeture) VALUES
('Cave de Paris', '12 rue de la Paix, Paris, France', NULL),
('Wine World', '45 High Street, London, UK', NULL),
('Enoteca Roma', 'Via Condotti, 25, Rome, Italy', '2025-01-01');

-- Remplissage de la table MouvementStock
INSERT INTO MouvementStock (idMagasin, idProduit, volume, recipient, date, dateCreation) VALUES
(1, 1, 750, 'bouteille', '2024-01-10', 100),
(1, 2, 750, 'bouteille', '2024-01-15', 50),
(2, 3, 750, 'bouteille', '2024-01-20', 30),

```

```
(2, 5, 330, 'canette', '2023-12-01', 500),
(3, 4, 700, 'bouteille', '2023-11-25', 20);
```

— *Remplissage de la table Vendeur*

```
INSERT INTO Vendeur (idMagasin, nom, salaire, estActif) VALUES
(1, 'Alice_Dupont', 2500.00, TRUE),
(2, 'John_Smith', 2200.00, TRUE),
(3, 'Giulia_Rossi', 2400.00, FALSE);
```

— *Remplissage de la table Client*

```
INSERT INTO Client (nom, adresse, email, pointDeFidelite, anneeNaissance) VALUES
('Paul_Morel', '34_avenue_des_Champs,_Paris,_France', 'paul.morel@example.com', 10, 1980),
('Anna_Garcia', '23_Carrer_Major,_Barcelona,_Spain', 'anna.garcia@example.com', 5, 1995),
('James_Taylor', '56_Broadway,_New_York,_USA', 'james.taylor@example.com', 15, 1978);
```

— *Remplissage de la table Vente*

```
INSERT INTO Vente (idMouvementStock, idVendeur, idClient) VALUES
(1, 1, 1),
(2, 1, 2),
(3, 2, 3);
```

— *Remplissage de la table Approvisionnement*

```
INSERT INTO Approvisionnement (idMouvementStock, dateCommande) VALUES
(4, '2023-11-01'),
(5, '2023-10-15');
```

— *Remplissage de la table Fournisseur*

```
INSERT INTO Fournisseur (nom, adresse, numeroTelephone) VALUES
('Distrib_Vins_France', '45_rue_du_Vin,_Bordeaux,_France', '+33_5_56_00_00_00'),
('Champagne_Select', '12_avenue_de_la_Champagne,_Reims,_France', '+33_3_26_00_00_00'),
('Global_Spirits', '99_Whiskey_Lane,_Dublin,_Ireland', '+353_1_00_00_03');
```

— *Remplissage de la table Approvisionnement_Fournisseur*

```
INSERT INTO Approvisionnement_Fournisseur (idMouvementStock, idFournisseur) VALUES
(4, 1),
(5, 3);
```

Description de l'application

L'application permet de gérer et de voir le contenu des stocks d'un magasin.